Automated Review Rating System

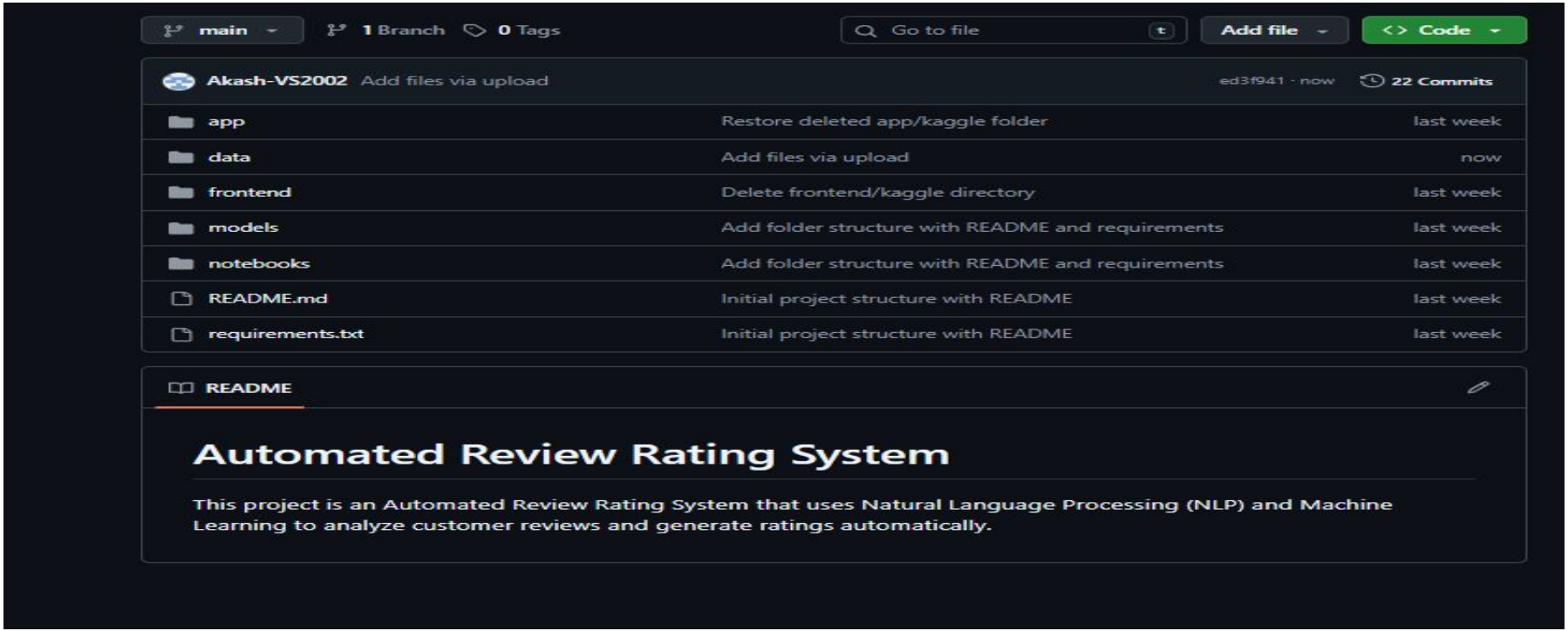
1 Project Overview

An intelligent system that analyzes customer reviews and predicts corresponding star ratings. It uses Natural Language Processing (NLP) to extract sentiment and key features from textual feedback. The model is trained on labeled review data to learn patterns between language and rating. This enables automated, consistent, and scalable review evaluation, helping businesses gain actionable insights and improve decision-making.

2 Environment Setup

- Python 3.9+ was used as the base language.
- Key libraries: Pandas, NumPy, Scikit-learn, Joblib, Matplotlib,Seaborn.
- IDE Used Jupyter Notebook
- Verified proper functioning of packages for data preprocessing, NLP, and data visualization.

3 GitHub Project Setup



4 Data Collection

Datasets Collected from Kaggle:

- https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews

  • Data includes user reviews and corresponding ratings (1–5 stars)

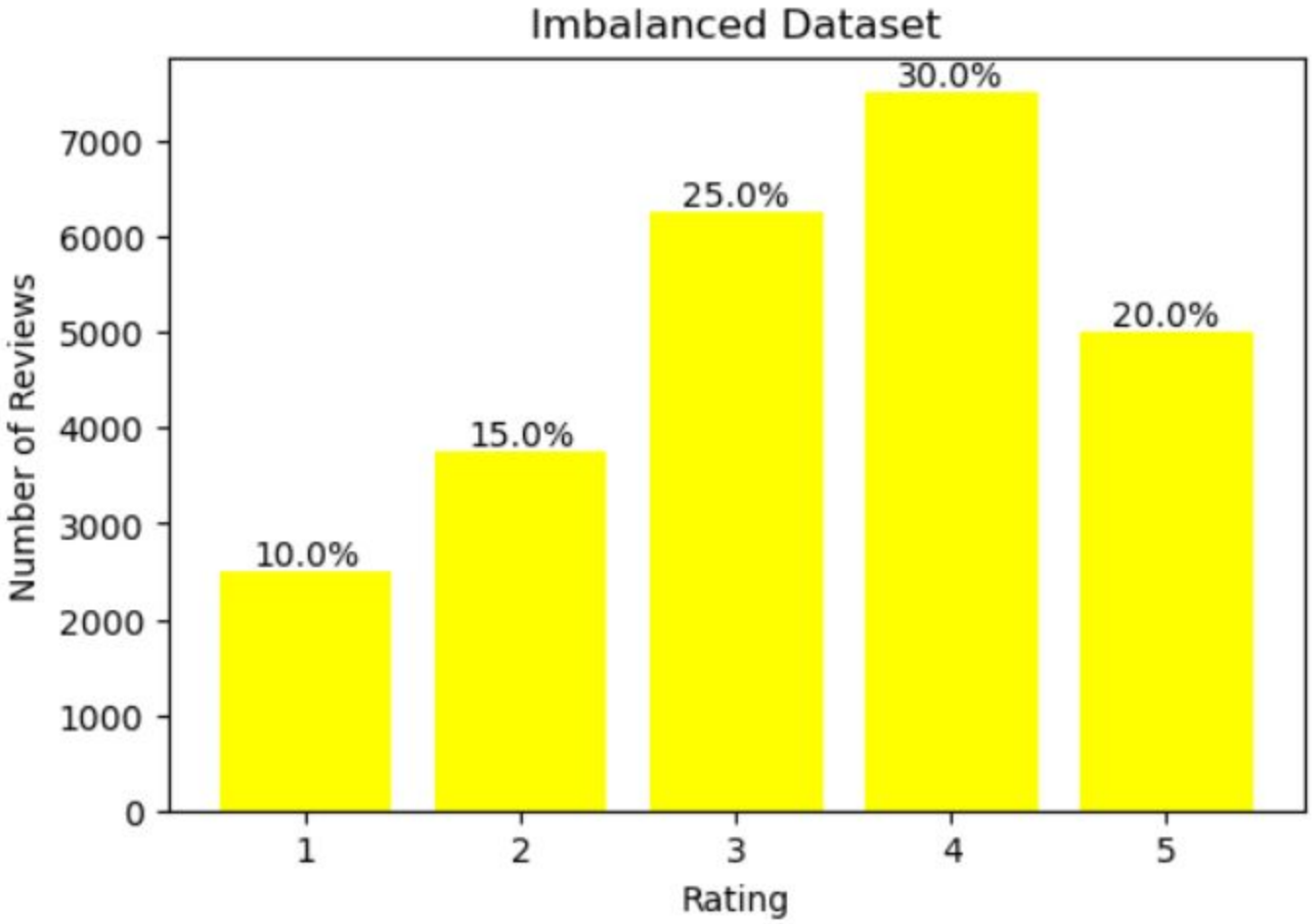
  • Ensured Imbalanced representation of ratings for better model training

4.1 Imbalancing the Dataset

The dataset contains **25,000 customer reviews**, distributed across five star ratings in an **imbalanced manner**. Unlike a balanced dataset, the reviews are unevenly spread:

- **1★:** 10% (2,500 samples)
- **2★:** 15% (3,750 samples)
- **3★:** 25% (6,250 samples)
- **4★:** 30% (7,500 samples)
- **5★:** 20% (5,000 samples)

imbalanced dataset 👉 Imbalanced_Dataset.csv


Imbalanced Dataset
Number of Reviews
0
1000
2000
3000
4000
5000
6000
7000
10.0%
15.0%
25.0%
30.0%
20.0%
1
2
3
4
5
Rating

5 Data Preprocessing

Data preprocessing involves cleaning, transforming, and organizing raw data to make it suitable for analysis or modeling. It ensures **accuracy, consistency, and better model performance** in machine learning tasks.

5.1 Removing Duplicates

Identify and remove duplicate reviews to avoid bias and redundancy in the dataset.

5.2 Detect and Remove Confusing Reviews

Detect and remove reviews where the text does not match the given star rating, ensuring label accuracy.

5.3 Dropping Unnecessary Columns

Remove columns from the dataset that do not contribute to review rating prediction.Reduces noise and improves model training efficiency.

Code 👉df = df.drop(columns=['review_id', 'username', 'date'])

5.4 Handling Missing Values

Identify and handle missing or null values in the dataset to ensure clean and complete data.Missing values in reviews or ratings can negatively affect model training and predictions.

Code 👉df = df.dropna()

5.5 Converting Text to Lowercase

Convert all review text to lowercase to maintain uniformity.Helps the model treat words like `Good` and `good` as the same token, reducing redundancy.

Code 👉df['review_text'] = df['review_text'].str.lower()

5.6 Removing URLs, Emojis, and Special Characters

Clean review text by removing unnecessary elements like URLs, emojis, and special characters.
Helps reduce noise and ensures that the text is suitable for feature extraction and modeling.

Code 👉 text = re.sub(r'http\S+|www.\S+', '', text), text = text.encode('ascii', 'ignore').decode('ascii'), text = re.sub(r'[^a-zA-Z0-9\s]', '', text)

5.7 Removing Punctuations

Remove punctuation marks (like . , ! ? ; :) from review text.Helps reduce noise and ensures that words are treated consistently during feature extraction.

Code 👉df['review_text'] = df['review_text'].str.replace(f"[{string.punctuation}]", "", regex=True)

5.8 Stopword Removal

Stopwords are common words (e.g., “the”, “is”, “and”) that do not contribute significant meaning to text analysis.

Removing them reduces noise and improves model performance for sentiment or review rating prediction.Total stopwords removed: 106161

Removed stopwords

['a', 'about', 'above', 'after', 'again', 'against', 'all', 'am', 'an', 'and', 'any', 'are', 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'd', 'did', 'do', 'does', 'doesn', 'doing', 'don', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'has', 'have', 'having', 'he', 'her', 'here', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'it', 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'more', 'most', 'my', 'myself', 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', 'she', 'should', 'so', 'some', 'such', 't', 'than', 'that', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'we', 'were', 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', 'y', 'you', 'your', 'yours', 'yourself', 'yourselves']

Code 👉

```
    def remove_stopwords(text):
    text = str(text).lower()
    text = re.sub(r'[^a-z\s]', '', text)
    words = text.split()
    cleaned_words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]
    return " ".join(cleaned_words)
df = pd.read_csv(r"C:/Users/MY PC/OneDrive/Desktop/kaggle/Imbalanced_Dataset.csv")
df['Cleaned_Review'] = df['Review'].apply(remove_stopwords)
```

5.9 Lemmatization

Lemmatization is the process of reducing words to their dictionary or base form (lemma). It considers the context and meaning of the word. Unlike simple stemming, lemmatization produces real words that exist in the language. This helps in preserving the semantic meaning of the text, which improves the accuracy of NLP tasks like sentiment analysis, text classification, and review rating prediction.

Example:

- Words like `running`, `ran`, `runs` → lemmatized to `run`
- Words like `better` → lemmatized to `good`

Why Lemmatization is Better than Stemming

Meaning Preservation:

- Lemmatization reduces words to their **real dictionary form (lemma)**, preserving the meaning.
- Stemming simply cuts word endings, which can produce non-words or incorrect forms.
  Accuracy:
- Lemmatization considers the **context and part of speech**, making it more accurate.
- Stemming uses a heuristic approach, often leading to errors.
  Example:
- Lemmatization: `running` → `run`, `better` → `good`
- Stemming: `running` → `run`, `better` → `better` (unchanged, may be incorrect)

5.10 Filter by Word Count

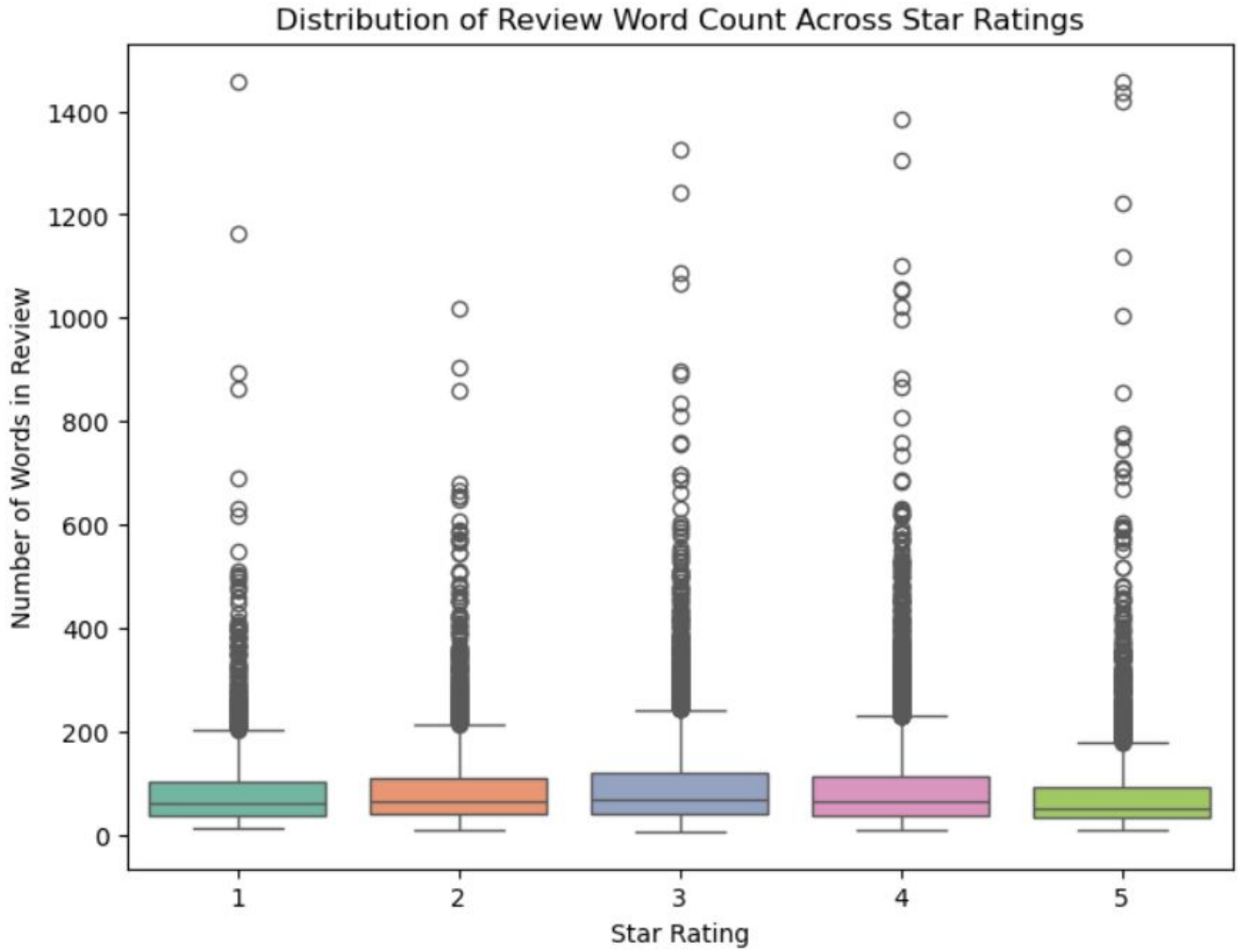
Filtering reviews by word count ensures that extremely short or empty reviews, which may not provide meaningful information, are removed.This improves model training by focusing on reviews with enough content to learn patterns.

6 Data Visualization

Data visualization helps to **understand the dataset**, identify patterns, and communicate insights effectively before model training. In the Automated Review Rating System, we visualize reviews, ratings, and text characteristics.

Box Plot

A **box plot** is a visual tool used to display the distribution of numerical data, showing the **median, quartiles, and potential outliers**. It helps quickly understand how data is spread and identify any extreme values. In this project, **box plots were used to compare the word count distribution of reviews across different star rating classes**, allowing us to analyze patterns such as whether longer reviews tend to correspond to higher or lower ratings.


Distribution of Review Word Count Across Star Ratings
Number of Words in Review
0
200
400
600
800
1000
1200
1400
1
2
3
4
5
Star Rating

Sample of 1-Star Rating

1. dont know received burger colorless mine red wonder safe eat
2. ive amazon since ive ordered item first review ive written read review item figured seller would learn try put many different sampler item opened package total kcups horrible pumpkin spice flavor couldnt get rid save life extremely upset feel ripped avoid seller cost got much better deal shoffeecom
3. product taste like expected sampling plum sauce like one would purchase item
4. last year another turducken great year market brought never buy one instruction state cook hour minute frozen took hour could lived took dry outside still raw middle trust market heard several complaint yearbr pa
5. received today wife horrible tasting coffee ever wasnt case caring coffee right nasty wife thought dont know got bad batch wont ordering

Sample of 2-Star Rating

1. nice convenient package hummus cracker good combo however content package gluten free chocolate nut mix produced equipment also process wheat celiac disease sensitive gluten two item completely gluten free
2. use whole lot little vanilla flavor good value vanilla extract work better everytimebr bottle beautiful practical wire top hard move order open bottle also shaker spoon metal cap hole held plastic cap underneath doesnt hole whats deal
3. excited try item calorie well sound good trueit doesnt taste anything like peanut whatsoever really wanted like unfortunatley wasted money dont waste
4. bought gum remember chewing kid thought juciy filling bestbr well see cant found store anymore hard liquid center like gel lost flavor within five minute really upset stuck case waste money
5. ok im deployed thought would buy something odd far beef jerky guy troop person liked jerky bad good

Sample of 3-Star Rating

1. disappointed product chewed gum get rubbery fast wouldnt recommend anyone else
2. make great waffle contrary description need add oil egg mix rather water probably issue people someone temporarily without kitchen fridge like u little disappointing
3. received order french lentil barry farm direction cooking lentil additional research know use something excited
4. kind healthy grain vanilla blueberry cluster flax seed better snack eat cereal milk taste acquired taste youve eaten start taste better never taste bad taste little like cardboard get used id recommend people health conscience like food like isnt something buy
5. dont know java one theyre managing put good coffee defective pod last shipment least one five pod sealedyou open envelope loose coffee edge pod open due opening incorrectly filter paper ripped damaged isnt glued sealed properly definitely recommended buy unless want waste money like coffee stay pod

Sample of 4-Star Rating

1. enjoyed soup yes would like fresh pho soup area live find vietnamese restaurant mile made soup added fresh bean sprout fresh cilantro fresh green onion strip cooked steak basic soup add thing itnot expect open strip steak going appear box recommend soup starter great soup adding fresh ingredient fresh thing never let

2. make week special treat there toasted coconut coconut powder crossed mind buy aunt jemima sp add shredded coconut tho sure would good probably worth try since canister pricey tho pricey bought bulk

3. way spicy love amys product feel like warning heat index

4. looking cereal white flour white sugar cereal taste great fill

5. nice bubbly beverage didnt satisfy desire soda lol yummy though

Sample of 5-Star Rating

1. bought lb merckens coco lite coating make chocolate covered strawberry valentine day tasted great melted well dried quickly strawberry great
2. began ordering using subscribe save program posted overseas back state probably continue buy amazon price way better grocery store around place compete walmart bit hike literally son love cereal mysteriously disappears much rapidly weekend husband eats breakfast home although healthier cereal happy one made whole grain relatively low sugar content especially compared sugary cereal available overseas believe u thanks amazon carrying product
3. fantastic people cant eat sugar ie diabetic youve scoured globe looking sugarfree snack doesnt fake sugar flavour search endedbr br huge haribo fan widely available u friend germany send haribo candy time biggest fear would live expectation relieved find barely tell differencebr br one word caution mentioned several reviewer cannot eat large quantity go sugar free candy stomach discomfort flatulencegas diarrhea wellknown side effect sugarfree candy wise eat dozen piece
4. husband lived spain year missing yummy food ate ordered cheese blown away great tasted husband loved real spanish cheese well ordering assortment
5. skeptical first actually feeling lot better finger joint also nail became healthy strong keeping french natural nail mixing soy protein shake morning breakfast doesnt strange taste sonperfect

7 Train–Test Split

The dataset is divided into two parts:

- **Training set (80%)** → used to train the model.
- **Testing set (20%)** → used to evaluate model performance on unseen data.

Stratified sampling is applied to ensure that all star rating classes (1★–5★) are proportionally represented in both training and testing sets, according to their **imbalanced distribution**:

- 1★ → 10%
- 2★ → 15%
- 3★ → 25%
- 4★ → 30%
- 5★ → 20%

This approach prevents the model from being biased toward any single rating class while still reflecting the real-world imbalanced distribution of reviews.

How It Was Done

The dataset contains **25,000 reviews** with an **imbalanced distribution** across five rating classes: 1★ (10%), 2★ (15%), 3★ (25%), 4★ (30%), and 5★ (20%). For effective model training and evaluation, the dataset was split into **20,000 training samples (80%)** and **5,000 testing samples (20%)**.

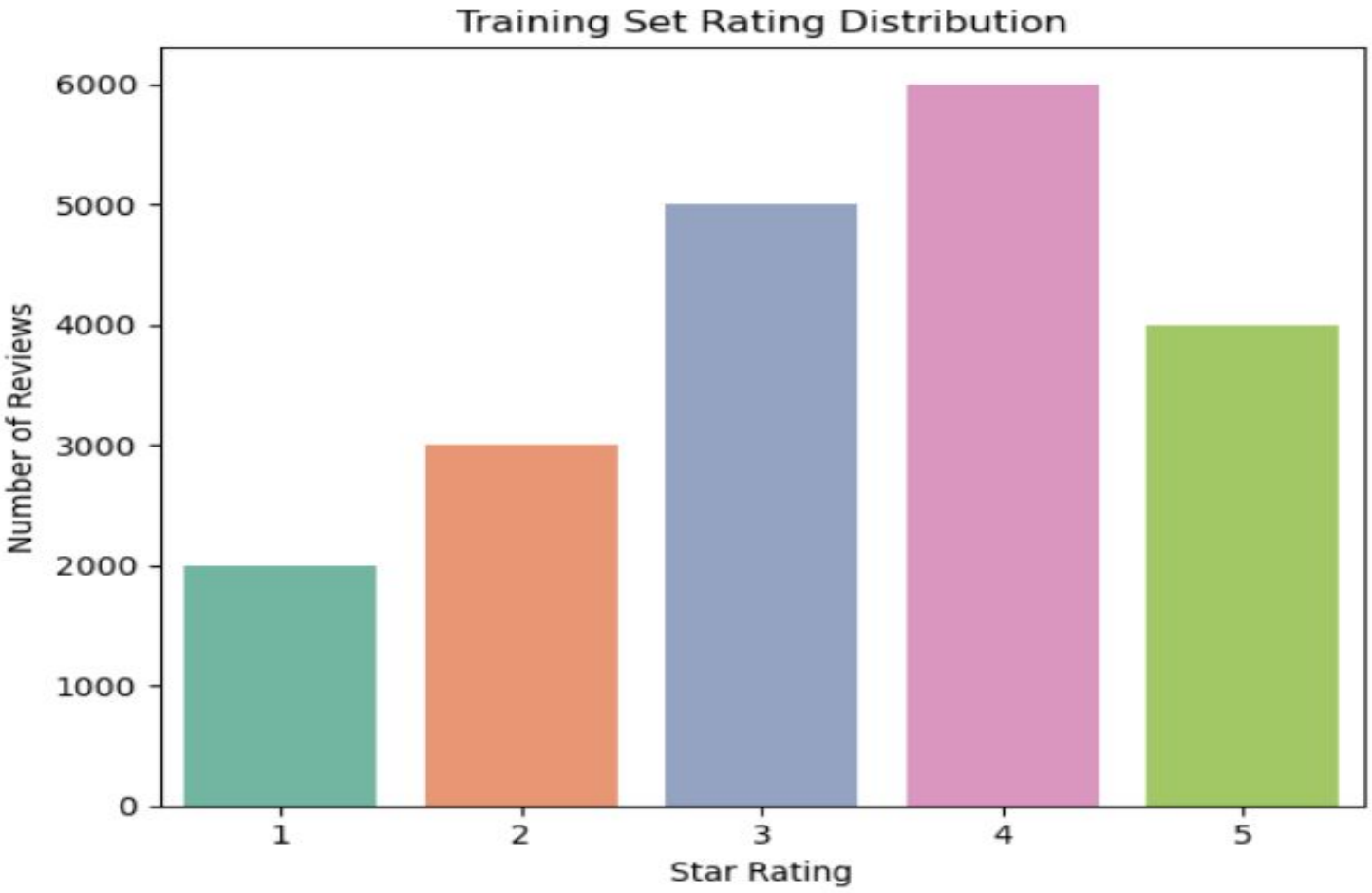
The `train_test_split` function was applied with the **stratify parameter**, ensuring that the imbalanced class proportions were preserved in both the training and testing sets. This maintained the natural imbalance of the dataset while preventing the model from being biased toward any particular rating class.

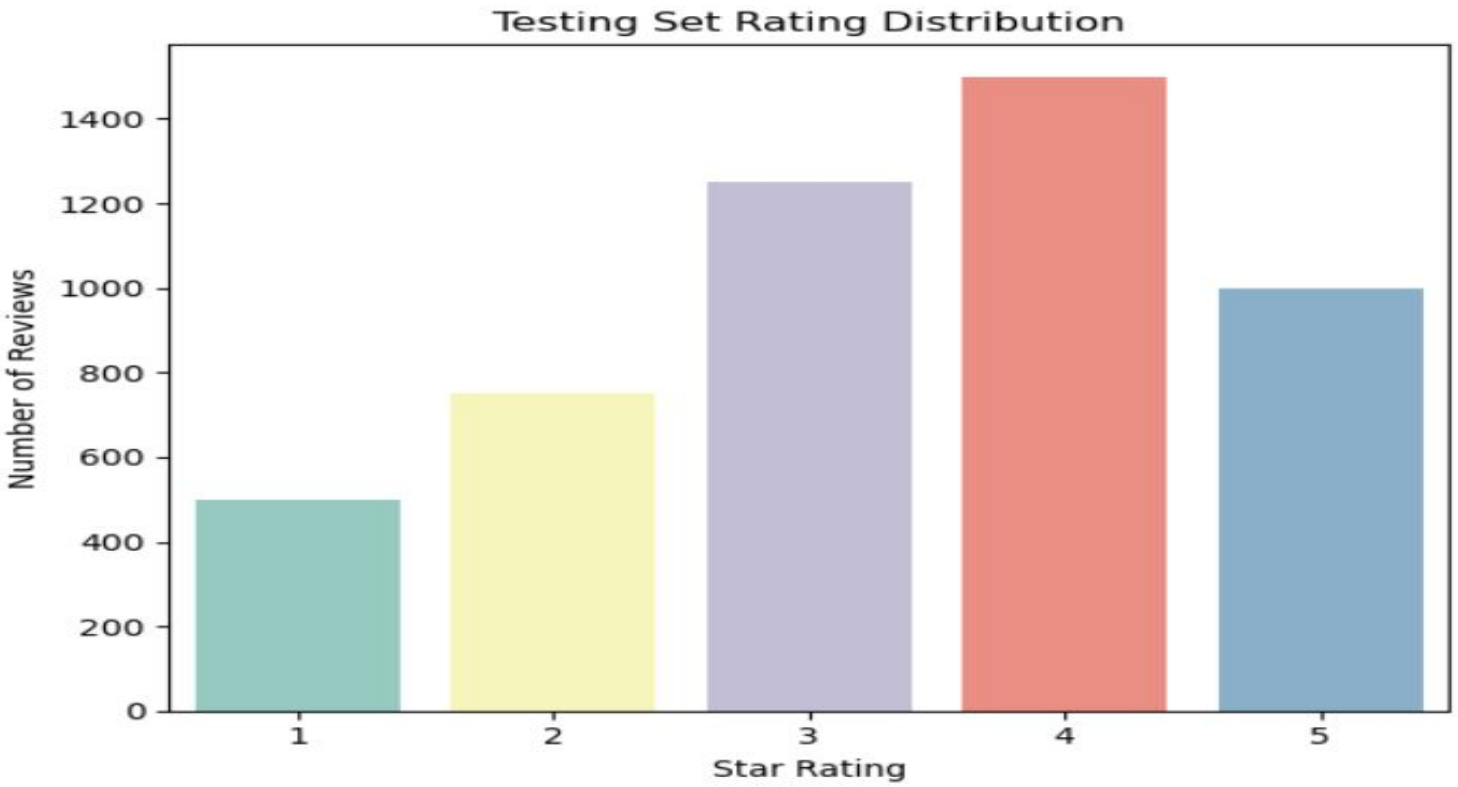
Training Dataset: imbalanced_train.csv

Testing Dataset: imbalanced_test.csv

Code 👉 df.columns = df.columns.str.strip()

```
# Features and target
X = df['Review']
y = df['Rating']
# Stratified train-test split (80%-20%)
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Combine features and target for saving
train_df = pd.DataFrame({'Review': X_train_text, 'Rating': y_train})
test_df  = pd.DataFrame({'Review': X_test_text,  'Rating': y_test})
```


Training Set Rating Distribution
Number of Reviews
0
1000
2000
3000
4000
5000
6000
1
2
3
4
5
Star Rating


Testing Set Rating Distribution
Number of Reviews
0
200
400
600
800
1000
1200
1400
1
2
3
4
5
Star Rating

8 Text Vectorization

Text vectorization transforms textual reviews into numerical representations that machine learning models can process. It converts words into features, capturing patterns and relationships within the text for effective model training.

TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF is a text vectorization technique that converts words into numerical values by combining **Term Frequency (TF)** and **Inverse Document Frequency (IDF)**. It highlights words that are important in a document while down-weighting common, less informative words.

Formula For TF-IDF

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF - IDF(t,d) = TF(t,d) * IDF(t)$$

Code 👉 X_train = tfidf.fit_transform(train_df['Review'])

X_test = tfidf.transform(test_df['Review'])

y_train = train_df['Rating']

y_test = test_df['Rating']

Output 👉

	ability	able	absolute	absolutely	absorb	absorbed	absorbs	acai	accept	acceptable	...	zero	zest	zevia	zico	zinc	zing	zinger	zip	ziploc	ziplock
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19995	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19996	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19997	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19998	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19999	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

20000 rows × 5000 columns

9 Model Training

Objective: Train multiple machine learning models on vectorized text data to predict user ratings. Evaluate and select the best-performing model.

Models Trained

- **Logistic Regression**
- **Linear SVM (LinearSVC)**
- **Multinomial Naive Bayes**
- **Decision Tree Classifier**
- **Linear Regression**

Code 👉models = {

```
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Linear SVM": LinearSVC(),
    "Multinomial NB": MultinomialNB(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier()
}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"--- {name} ---")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))
```

Model Name	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Logistic Regression	0.4406	0.44	0.4	0.41
Linear Regression	0.2446	nan	nan	nan
SVM	0.375	0.36	0.36	0.36
Naive Bayes	0.3702	0.57	0.27	0.22
Decision Tree	0.3178	0.3	0.29	0.29

9.1 Model Fine-Tuning

“Model fine-tuning optimizes hyperparameters to improve predictive performance, reduce overfitting, and enhance generalization. GridSearchCV systematically tests parameter combinations, selecting the best model configuration for higher accuracy and F1-score.”

Code 👉param_grid_lr = {

'C': [0.1, 1, 10],

'penalty': ['l2'],

'solver': ['lbfgs', 'saga']

}grid_lr = GridSearchCV(LogisticRegression(max_iter=1000), param_grid_lr, cv=5, scoring='f1_macro')

grid_lr.fit(X_train, y_train)

best_lr = grid_lr.best_estimator_

Output 👉📊 Best Model for IMBALANCED Dataset:

Logistic Regression: Accuracy=0.4324 | Best Param=1

9.2 Why Logistic Regression Was Chosen

Highest Accuracy:
Logistic Regression achieved **0.4406**, higher than all other models, making it the most reliable overall predictor.
Balanced Class Performance:

- Macro F1-score **0.41** is the highest among tested models.
- It predicts both minority and majority classes better than Naive Bayes or Random Forest.
  Better Generalization:
- Less overfitting compared to Decision Tree and Random Forest.
- Handles sparse TF-IDF features efficiently, unlike SVM which can be sensitive to parameters.
  Stable and Interpretable:
- Coefficients provide insights on which words affect ratings.
- Fast training makes it suitable for iterative fine-tuning.

9.3 Final Model Prediction

S.No	Actual Rating	Predicted Rating	Review (First 200 chars)
1	1	1	stella doro company sold baking believe north although recipe may machine used use cake mixer destroy texture called company purchasing anisette biscuit awful sandy texture rather company apologized s...
2	1	4	around long time using profit ultra right wing
3	1	3	really enjoy pumpkin spice latte wanted try recreate looked around review settled one since everyone gave candy hot syrup aftertaste flavor hot cinnamon pumpkin flavor safety cap mine work ring cap br...
4	1	3	poison two beautiful saint using began routine would go outside vomit supposed morning treat dental investigated chemical package warns stopped using ignorantly designed dog hating sick anyone like do...
5	1	1	trusted quaker oat company bring safe soon received box crispy oat cookie bar fruit pulled blueberry cookie bar devoured nice ratio crunch sweet filling thought would good snack keep minute started fe...
6	2	3	pineapple strawberry aloe vera king much better banana ok flavor taste bit hate buy
7	2	3	unnecessarily loaded sugar dust make lack intrinsic seen turkish delight bad quality sale less buck local
8	2	2	imagine surprise walking dog picked poo one walking nearest garbage leaving trail behind u bottom split oh land go back poo another would upset unfortunately happened first box mean walk mind spending...
9	2	2	okay gf buy taste alright really dry
10	2	3	using organic dry dog food godsend discovered little jack severe allergy tormented day night unrelieved trying endless avenue find relief point thought might put find idea came maybe chemical decided ...
11	3	4	basically title sum like really wish something also work ice cube tray one lid put large baggie putting freezer cover aluminum spray tray pam really make frozen food pop right one nice thing tray ice ...
12	3	3	two three cat turned nose flavor pill used conceal tapeworm pill half careful medicine residue hand sealed one cat gobbled two cat decided give grief ended get pill another love idea product impossibl...
13	3	2	really wanted like sour love trolli sour gave try reading positive agree ana worm way sour smell wonderful beat price like extremely sour gummy
14	3	4	order came inspection tea clean impression bland realize rooibus blend sure tea old nature use basis good citrus candied like little sugar stevia use
15	3	2	dental stick somewhat long white stick appealing two small consume deperate buy
16	4	4	pretty good remember wish tad softer would gotten
17	4	3	little expensive even subscribe would buy regular one store around fan cranberry okay
18	4	4	expecting something bit salty bar definitely peanut butter layer bottom spending morning truck melted would recommend storing warm place lunchbox unless near ice pack melted peanut butter coated insid...
19	4	4	wild sweet orange herbal infusion nice orange herbal contains blackberry citric rose spearmint natural orange hibiscus rose natural orange ginger root licorice subtle wonderful tea enjoy orangey inter...
20	4	4	good tasting popcorn plenty one watch microwave burn otherwise good price good
21	5	4	squeezed water liquid mixed water little use much little second drink refreshing overly taste like enjoyed plan getting
22	5	5	love yerba mate better chai tea go well soy seller offer best price fast highly
23	5	5	first found year old go gluten free little worried would handle gf bisquick made lot able make favorite food chicken corn pizza chinese sour pancake also big hit quick easy recipe take chicken finger ...
24	5	4	double bergamot earl grey tea make nice cup stand well used office economical
25	5	5	excellent product healthy combination whole food creating proper nutrition dog without garbage many

10 Model Testing on Cross Test Sets

Evaluate pre-trained models on **cross test sets** to understand how **training data distribution (balanced vs imbalanced)** affects performance.

10.1 Load Model & Test Data

Code 👉Model_B = joblib.load('Model_B.pkl')

```
test_bal = pd.read_csv('test.csv')  # balanced test
X_test_bal = test_bal['Review'].fillna("no review").astype(str)
y_test_bal = test_bal['Rating']
```

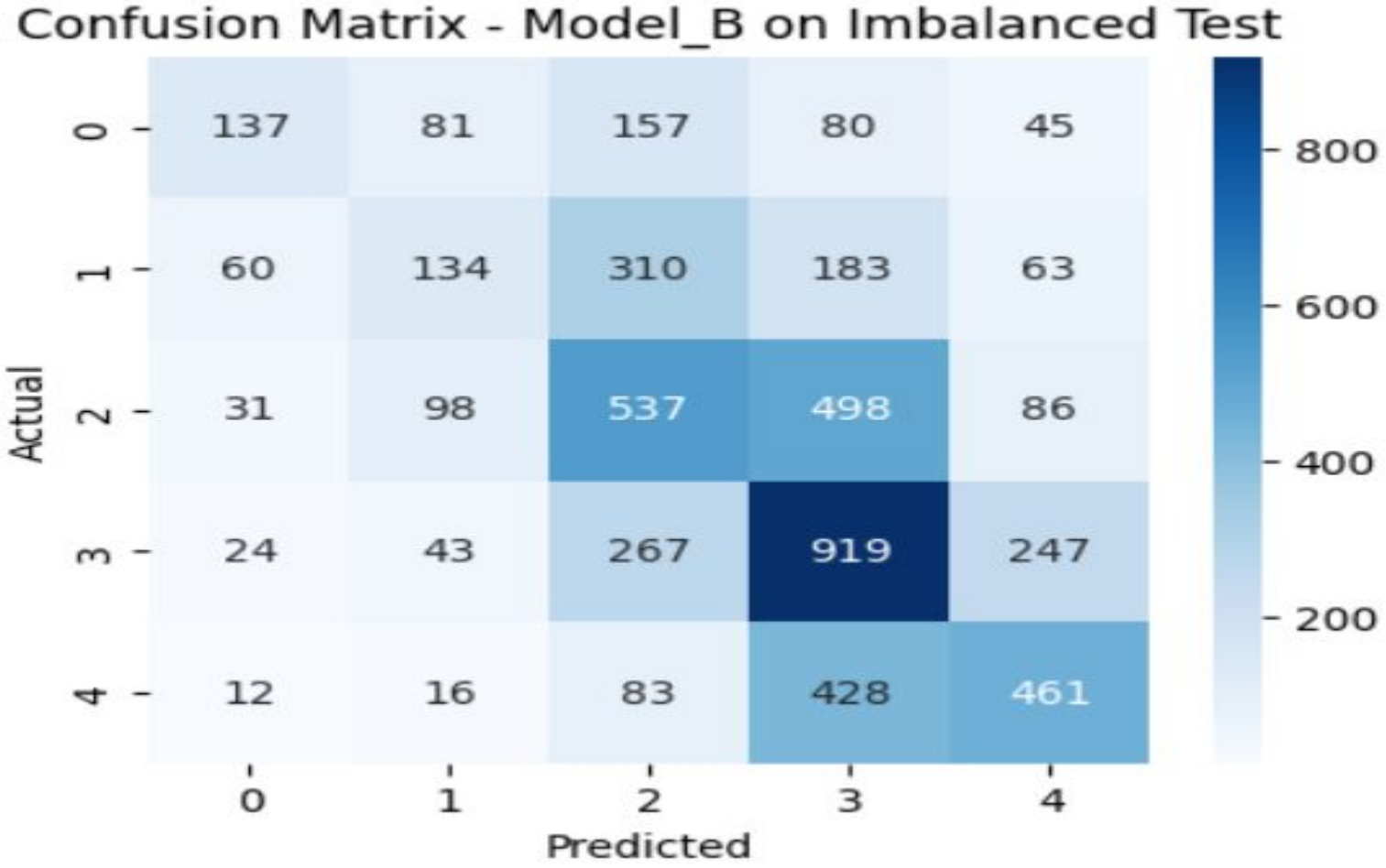
10.2 Evaluate Performance

Code 👉def evaluate_model(y_true, y_pred, model_name):

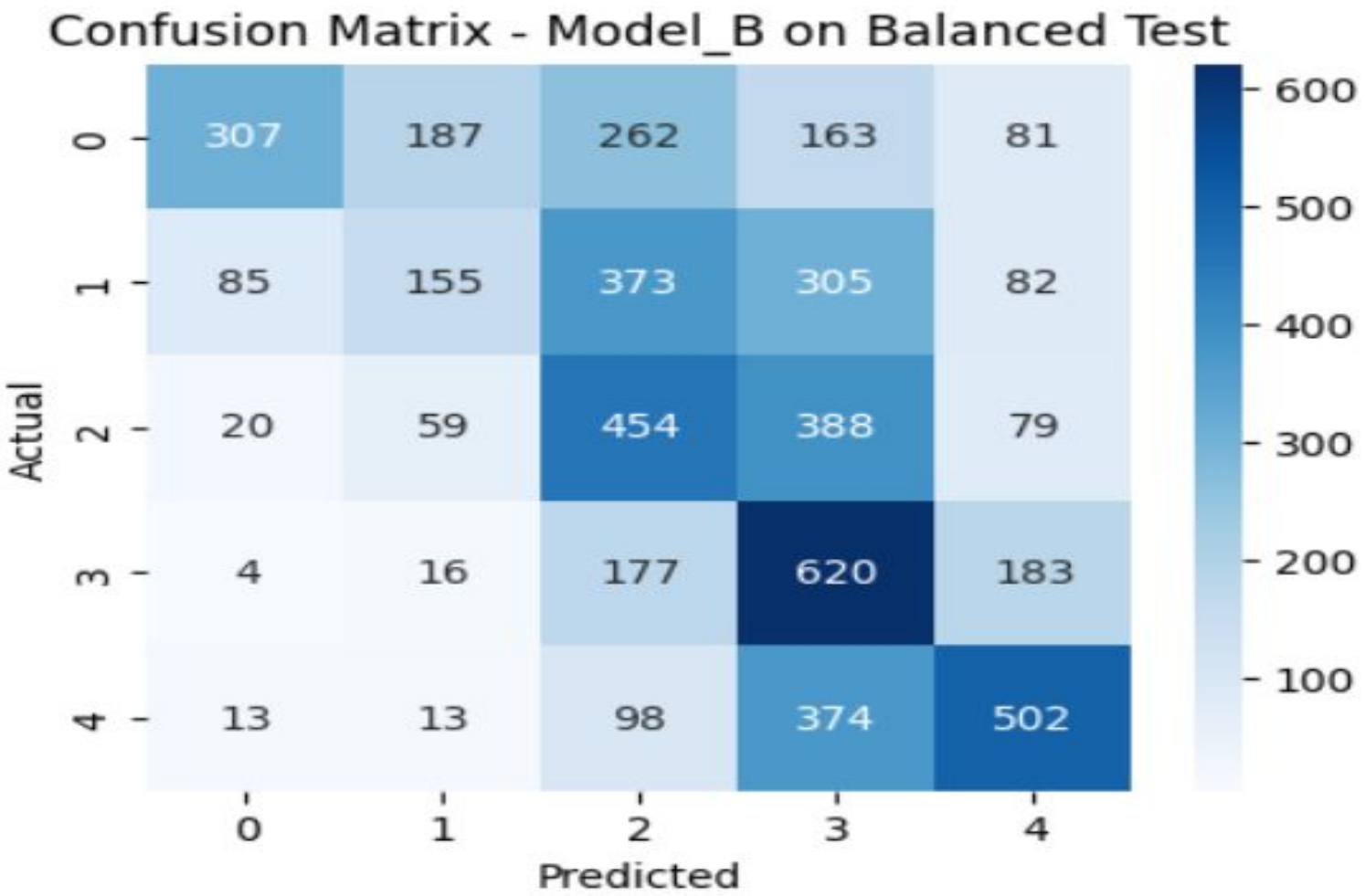
```
    acc = accuracy_score(y_true, y_pred)
    report = classification_report(y_true, y_pred, output_dict=True)
    cm = confusion_matrix(y_true, y_pred)
acc_B_bal, report_B_bal, cm_B_bal = evaluate_model(
    Model_B, X_test_bal, y_test_bal, "Model_B on Balanced Test"
)
```

Result

```
=== Model_B on Imbalanced Test ===
Accuracy: 0.4376
Classification Report:
              precision    recall  f1-score   support

           1       0.52      0.27      0.36       500
           2       0.36      0.18      0.24       750
           3       0.40      0.43      0.41      1250
           4       0.44      0.61      0.51      1500
           5       0.51      0.46      0.48      1000

    accuracy                           0.44      5000
   macro avg       0.44      0.39      0.40      5000
weighted avg       0.44      0.44      0.42      5000

Confusion Matrix:
```


Confusion Matrix - Model_B on Imbalanced Test
Actual
Predicted
0
1
2
3
4
137 81 157 80 45
60 134 310 183 63
31 98 537 498 86
24 43 267 919 247
12 16 83 428 461
800
600
400
200

```
=== Model_B on Balanced Test ===
Accuracy: 0.4076
Classification Report:
              precision    recall  f1-score   support

           1       0.72      0.31      0.43      1000
           2       0.36      0.15      0.22      1000
           3       0.33      0.45      0.38      1000
           4       0.34      0.62      0.44      1000
           5       0.54      0.50      0.52      1000

    accuracy                           0.41      5000
   macro avg       0.46      0.41      0.40      5000
weighted avg       0.46      0.41      0.40      5000
```


Confusion Matrix - Model_B on Balanced Test
Actual
Predicted
0
1
2
3
4
307
187
262
163
81
85
155
373
305
82
20
59
454
388
79
4
16
177
620
183
13
13
98
374
502
600
500
400
300
200
100